

Modélisation numérique de la diffusion non linéaire et application à la croissance tumorale

December 5, 2025

1 Diffusion non linéaire et influence du paramètre β

1.0.1 Application à un modèle simplifié de croissance tumorale

Objectif : implémenter pas à pas un schéma volumes finis implicite en 1D pour l'équation de diffusion rapide, et étudier le comportement des entropies discrètes au cours du temps.

1.1 Imports et paramètres globaux

```
[3]: import numpy as np
import matplotlib.pyplot as plt

# =====
# Paramètres globaux
# =====

beta = 0.5          # cas de base : diffusion rapide ( = 1/2)
L = 2.0             # longueur du domaine [-1,1]
N = 200             # nombre de volumes de contrôle
dx = L / N
T = 0.1             # temps final
dt = 2e-4
M = int(T / dt)     # nombre de pas de temps

# Grille : centres de mailles sur [-1,1]
x = -1.0 + (np.arange(N) + 0.5) * dx

print(f"N = {N}, dx = {dx:.5f}, dt = {dt}, M = {M}")
```

N = 200, dx = 0.01000, dt = 0.0002, M = 500

1.2 Commentaire : Paramètres globaux et discrétisation spatiale

On considère l'équation de diffusion non linéaire en dimension 1 :

$$u_t = \partial_{xx}(u^\beta),$$

sur l'intervalle $[-1, 1]$ avec conditions de Neumann homogènes.

Les paramètres numériques sont :

- $N = 200$: nombre de mailles du maillage,
- $dx = \frac{L}{N} = 0.01$: pas spatial,
- $dt = 2 \times 10^{-4}$: pas de temps,
- $T = 0.1$: temps final,
- $M = 500$: nombre total de pas de temps,
- $\beta = 0.5$: régime de diffusion rapide.

La grille spatiale est définie aux **centres des volumes finis**, ce qui est cohérent avec un schéma de type volumes finis.

On travaille ici dans le régime de *fast diffusion* car $\beta < 1$.

1.3 Condition initiale + intégrale sur $[-1, 1]$

```
[4]: # =====
# Condition initiale u0
# =====

def u0(x, x0=-0.3, x1=0.3, C=3000.0):
    """
    Définit la condition initiale u0(x) sous la forme d'une bosse polynomiale
    compacte supportée sur l'intervalle [x0, x1].

    - x : tableau des points de grille
    - x0 : borne gauche du support de la bosse
    - x1 : borne droite du support de la bosse
    - C : constante de normalisation (amplitude)

    Formellement :
        u0(x) = C * ((x0 - x) * (x - x1))_+^2
    où (·)_+ désigne la partie positive (max(·, 0)).
    """
    # Produit (x0 - x)(x - x1) :
    # > 0 si x ∈ (x0, x1), < 0 sinon.
    z = (x0 - x) * (x - x1)
    # On ne garde que la partie positive : en dehors de (x0, x1), z devient 0.
    z = np.maximum(z, 0.0)
    # On élève z au carré et on multiplie par C : bosse lisse, nulle hors [x0,
    ↪x1].
    return C * z**2

# On évalue la condition initiale sur la grille x
u_init = u0(x)

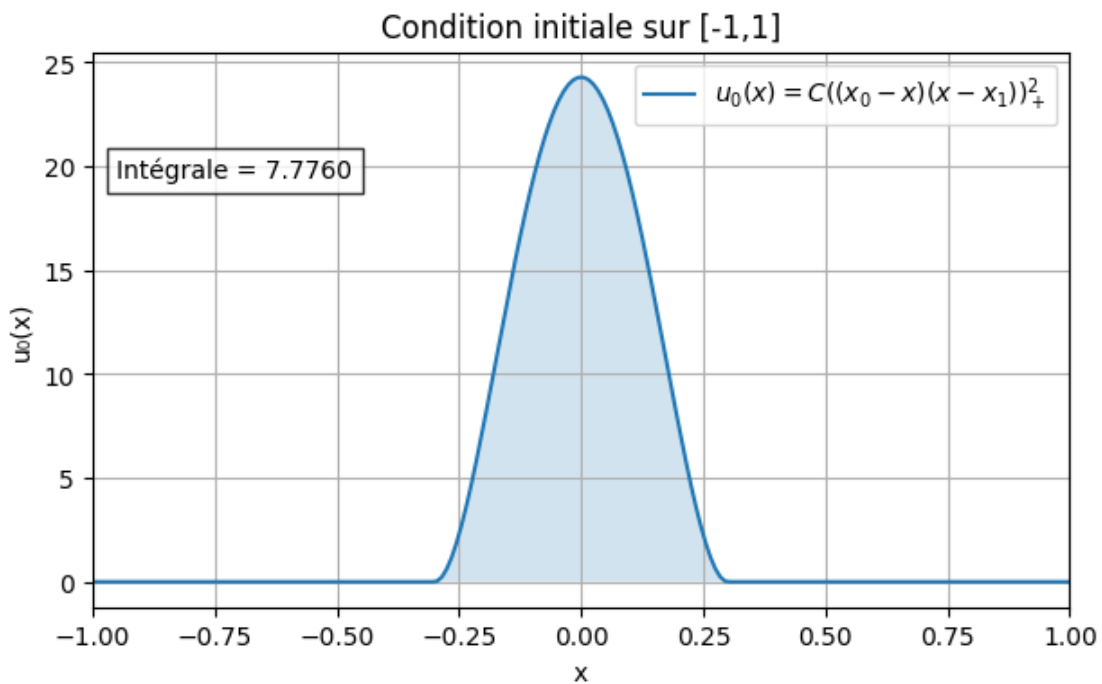
# Intégrale numérique de u0 sur [-1, 1] (approximation de ∫ u0(x) dx)
```

```

# Ici, np.trapz utilise la méthode des trapèzes sur la grille x.
integrale_u0 = np.trapz(u_init, x)

# Tracé de la condition initiale
plt.figure(figsize=(7,4))
plt.plot(x, u_init, label=r"$u_0(x) = C((x_0-x)(x-x_1))_+^2$")
# On colore la zone sous la courbe pour visualiser l'intégrale
plt.fill_between(x, u_init, alpha=0.2)
plt.xlim([-1, 1])
plt.xlabel("x")
plt.ylabel("u (x)")
plt.title("Condition initiale sur [-1,1]")
plt.grid(True)
# On affiche la valeur numérique de l'intégrale directement sur la figure
plt.text(-0.95, 0.8*np.max(u_init),
        f"Intégrale = {integrale_u0:.4f}",
        bbox=dict(facecolor='white', alpha=0.8))
plt.legend()
plt.show()

```



1.4 Commentaire : Condition initiale

La condition initiale est définie comme :

$$u_0(x) = C ((x_0 - x)(x - x_1))_+^2, \quad \text{avec } x_0 = -0.3, x_1 = 0.3.$$

Cette fonction possède les propriétés suivantes :

- $u_0(x) \geq 0$ pour tout x ,
- $u_0(x) = 0$ hors de l'intervalle $[-0.3, 0.3]$,
- u_0 est symétrique par rapport à $x = 0$,
- u_0 modélise une bosse localisée représentant une concentration initiale.

On calcule ensuite la masse initiale :

$$M(0) = \int_{-1}^1 u_0(x) dx.$$

Numériquement, on obtient :

$$M(0) \approx 7.776,$$

ce qui coïncide avec la valeur analytique :

$$\int u_0(x) dx = \frac{C(x_1 - x_0)^5}{30} = 7.776.$$

Cela valide la discrétisation spatiale et le calcul numérique de l'intégrale.

1.5 Schéma implicite : Newton + tridiagonale

```
[5]: # =====
# Schéma Euler implicite + Newton en 1D
# =====

def residual_and_jacobian(u_new, u_old, beta, dx, dt, eps=1e-12):
    """
    Construit :
    - le résidu R(u_new)
    - les diagonales de la matrice jacobienne (tridiagonale)

    pour le schéma volumes finis implicite appliqué à :
        u_t = (u^~)_xx
    avec conditions de Neumann (flux nul aux bords).

    Paramètres
    -----
    u_new : tableau
        Approximation actuelle de u^~{n+1} (itérée de Newton).
    u_old : tableau
        Solution au pas de temps précédent u^~n.
```

```

beta : float
    Exposant dans l'équation de diffusion.
dx : float
    Pas de maille en espace.
dt : float
    Pas de temps.
eps : float
    Petit décalage pour éviter les puissances de zéro/négatif.
"""
N = u_new.size
# Coefficient diffusif global  $dt/dx^2$ 
D = dt / dx**2

# Résidu (taille N) et diagonales de la matrice tridiagonale
r = np.zeros(N) # résidu R(u_new)
main = np.ones(N) # diagonale principale
lower = np.zeros(N-1) # sous-diagonale
upper = np.zeros(N-1) # sur-diagonale

# éviter les puissances sur des valeurs négatives :
# on force u_new >= 0 (numériquement) et on ajoute eps
u_eps = np.maximum(u_new, 0.0) + eps
pow_beta = u_eps**beta # (u_new)^
d_beta = beta * u_eps**(beta - 1.0) # dérivée de u^ par rapport à u

# ----- Bord gauche (Neumann) -----
# Discrétisation de (u^)_xx à gauche avec flux nul au bord :
# on obtient une formule spéciale pour la première cellule.
r[0] = u_new[0] - u_old[0] + D*(pow_beta[0] - pow_beta[1])
main[0] = 1.0 + D*d_beta[0]
upper[0] = -D*d_beta[1]

# ----- Cellules intérieures -----
# Pour i = 1,...,N-2 on a un stencil symétrique :
#  $u_{n+1}^i - u_n^i + D(2u_i^{\sim} - u_{i-1}^{\sim} - u_{i+1}^{\sim}) = 0$ 
for i in range(1, N-1):
    r[i] = u_new[i] - u_old[i] + D*(2*pow_beta[i] - pow_beta[i-1] -
    ↪ pow_beta[i+1])
    main[i] = 1.0 + 2*D*d_beta[i]
    lower[i-1] = -D*d_beta[i-1]
    upper[i] = -D*d_beta[i+1]

# ----- Bord droit (Neumann) -----
# Même principe qu'à gauche, avec flux nul au bord droit.
r[N-1] = u_new[N-1] - u_old[N-1] + D*(pow_beta[N-1] - pow_beta[N-2])
main[N-1] = 1.0 + D*d_beta[N-1]
lower[N-2] = -D*d_beta[N-2]

```

```

return r, main, lower, upper

def solve_tridiag(main, lower, upper, rhs):
    """
    Résout un système linéaire tridiagonal  $Ax = rhs$ 
    par l'algorithme de Thomas (forward/backward substitution).

    A est représentée par :
    - main : diagonale principale
    - lower : sous-diagonale
    - upper : sur-diagonale
    """
    n = len(main)
    # On copie les tableaux pour ne pas modifier les entrées d'origine
    a = lower.copy()
    b = main.copy()
    c = upper.copy()
    d = rhs.copy()

    # ----- Phase de forward elimination -----
    for i in range(1, n):
        w = a[i-1] / b[i-1]      # coefficient d'élimination
        b[i] -= w * c[i-1]      # mise à jour de la diagonale principale
        d[i] -= w * d[i-1]      # mise à jour du second membre

    # ----- Phase de backward substitution -----
    x = np.zeros_like(d)
    x[-1] = d[-1] / b[-1]      # dernière inconnue
    for i in range(n-2, -1, -1):
        x[i] = (d[i] - c[i] * x[i+1]) / b[i]

    return x

def one_time_step(u_old, beta, dx, dt, max_iter=50, tol=1e-10, damping=0.5):
    """
    Effectue un pas de temps d'Euler implicite pour  $u_t = (u^*)_x$ 
    en résolvant le système non linéaire par Newton amorti.

    Paramètres
    -----
    u_old : tableau
        Solution au temps  $t^n$ .
    beta : float
        Exposant de la diffusion.

```

```

dx, dt : float
    Pas spatial et pas de temps.
max_iter : int
    Nombre maximal d'itérations de Newton.
tol : float
    Tolérance sur la norme infinie du résidu pour arrêter Newton.
damping : float
    Facteur d'amortissement (0 < damping ≤ 1) pour stabiliser Newton.

Retour
-----
u_new : tableau
    Approximation de la solution au temps  $t^{n+1}$ .
"""
# Initialisation de Newton : on part de u_old comme première approximation
u_new = u_old.copy()
for it in range(max_iter):
    # Calcul du résidu et du jacobien (sous forme de diagonales)
    r, main, lower, upper = residual_and_jacobian(u_new, u_old, beta, dx, dt)

    norm_r = np.linalg.norm(r, np.inf) # norme infinie du résidu

    # Critère d'arrêt de Newton : résidu suffisamment petit
    if norm_r < tol:
        # print(f"Newton convergé en {it} itérations (||r||∞ = {norm_r:.2e})")
        break

    # Résolution du système linéaire J * delta = -r
    delta = solve_tridiag(main, lower, upper, -r)

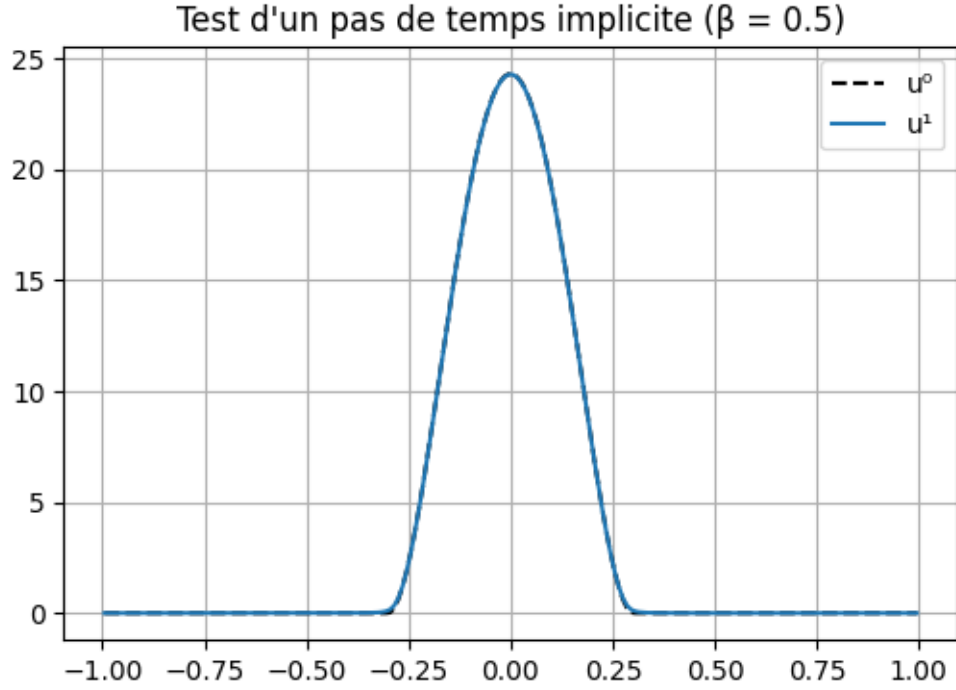
    # Mise à jour amortie + projection sur R pour garantir u ≥ 0
    u_new = np.maximum(u_new + damping * delta, 0.0)

return u_new

# Petit test sur un pas de temps pour vérifier visuellement le schéma implicite
u_test = one_time_step(u_init, beta, dx, dt)

plt.figure(figsize=(6,4))
plt.plot(x, u_init, 'k--', label="u ") # solution initiale
plt.plot(x, u_test, label="u¹") # solution après un pas
plt.legend()
plt.title("Test d'un pas de temps implicite (Δt = 0.5)")
plt.grid(True)
plt.show()

```



1.6 Commentaire : Schéma numérique et méthode de Newton

On résout l'équation :

$$u_t = (u^\beta)_{xx}$$

par une discrétisation en espace de type volumes finis et un schéma d'Euler implicite en temps.

À chaque pas de temps, on doit résoudre :

$$u^{n+1} - u^n + \frac{dt}{dx^2} A(u^{n+1}) = 0,$$

où le terme non linéaire dépend de u^{n+1} via u^β .

1.6.1 Construction du résidu et du jacobien

La fonction `residual_and_jacobian` calcule :

- le vecteur résidu $R(u^{n+1})$,
- la matrice jacobienne tridiagonale associée.

Les conditions de Neumann homogènes (flux nul) sont intégrées au bord via l'annulation du flux sortant.

Pour éviter des puissances de nombres négatifs, on définit :

$$u_\varepsilon = \max(u, 0) + \varepsilon.$$

Cela garantit la stabilité numérique lorsque $\beta < 1$.

1.6.2 Méthode de Newton amortie

La fonction `one_time_step` implémente une méthode de Newton :

- résolution du système tridiagonal par l'algorithme de Thomas,
- amortissement pour améliorer la convergence,
- projection sur $\mathbb{R}^+$:

$$u^{n+1} \leftarrow \max(u^{n+1}, 0).$$

Cela impose numériquement la **positivité de la solution**.

1.6.3 Test sur un pas de temps

On compare la solution initiale u^0 et la solution après un pas u^1 .

On observe que :

- les deux courbes sont presque confondues,
- la variation est très faible car dt est petit,
- le schéma est stable dès le premier pas.

Ce test permet de valider l'implémentation.

1.7 Flux de diffusion + graphe u vs flux

```
[9]: # =====  
# Flux de diffusion F_{i+1/2}  
# =====  
  
def flux_diffusion(u, beta, dx):  
    """  
    Calcule les flux de diffusion aux interfaces (Neumann : flux nul aux bords).  
    F_{i+1/2} = - ( (u~beta)_i - (u~beta)_{i-1} ) / dx  
    Retourne F de taille N+1 (interfaces de 0 à N).  
    """  
    # Récupération du nombre de points de discrétisation  
    N = len(u)  
  
    # Calcul de u~beta en s'assurant que u est positif (évite les problèmes_␣  
    ↪ avec beta non-entier)
```

```

u_beta = np.maximum(u, 0.0)**beta

# Initialisation du tableau de flux aux interfaces (N+1 points)
F = np.zeros(N+1)

# Calcul du flux aux interfaces intérieures (de 1 à N-1)
# Formule :  $F_{\{i+1/2\}} = - (u\_beta[i] - u\_beta[i-1]) / dx$ 
for i in range(1, N):
    F[i] = - (u_beta[i] - u_beta[i-1]) / dx

# Conditions aux bords de Neumann : flux nul aux extrémités
F[0] = 0.0 # Flux nul à gauche (x = -1)
F[N] = 0.0 # Flux nul à droite (x = +1)

return F

# Calcul du flux de diffusion initial avec la condition initiale u
F0 = flux_diffusion(u_init, beta, dx)

# Création des coordonnées des interfaces (décalées de dx/2 par rapport aux
↪centres)
x_interfaces = -1.0 + np.arange(N+1)*dx

# =====
# Visualisation de u et du flux F
# =====
plt.figure(figsize=(7,4))

# Tracé de la condition initiale u(x) aux centres des cellules
plt.plot(x, u_init, label="u (x) (centres)")

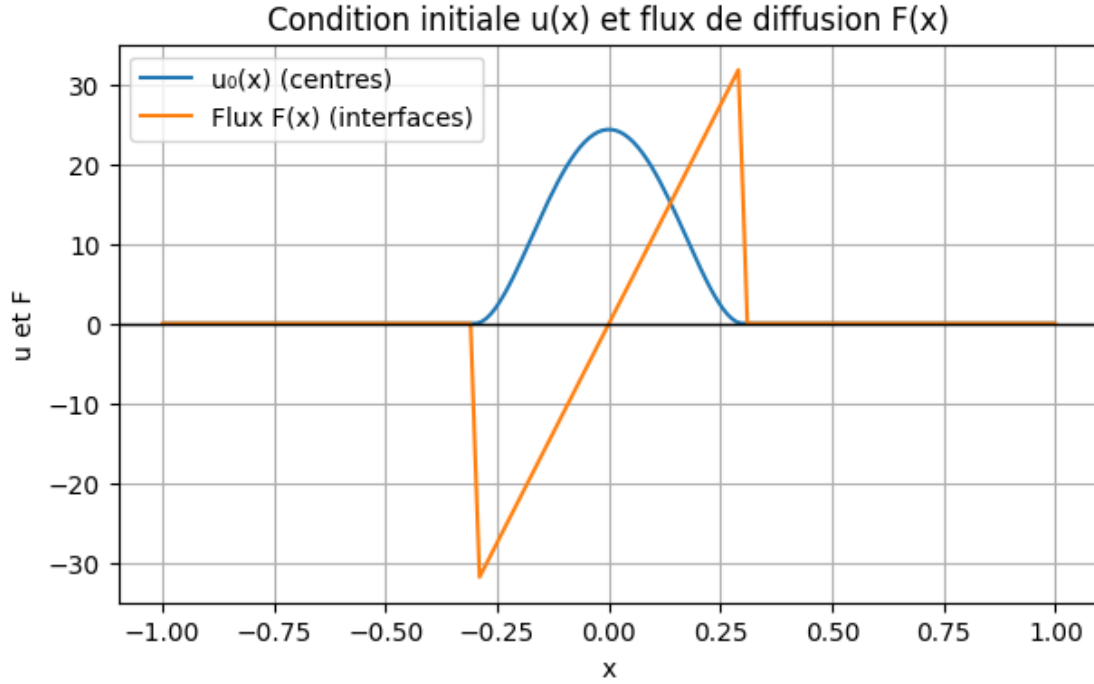
# Tracé du flux F(x) aux interfaces des cellules
plt.plot(x_interfaces, F0, label="Flux F(x) (interfaces)")

# Ligne horizontale à y=0 pour référence
plt.axhline(0.0, color="k", linewidth=1)

# Labels et titre
plt.xlabel("x")
plt.ylabel("u et F")
plt.title("Condition initiale u(x) et flux de diffusion F(x)")

# Affichage de la grille et de la légende
plt.grid(True)
plt.legend()
plt.show()

```



1.8 Commentaire : Flux de diffusion

On définit le flux de diffusion aux interfaces des cellules par :

$$F_{i+\frac{1}{2}} = -\frac{(u^\beta)_i - (u^\beta)_{i-1}}{dx}.$$

Des conditions de Neumann homogènes (flux nul) sont imposées aux bords :

$$F_{1/2} = F_{N+1/2} = 0.$$

1.8.1 Interprétation du flux

Le flux est lié à la pente de u^β :

$$F = -\partial_x(u^\beta).$$

Ainsi :

- si u^β augmente, alors $F < 0$,
- si u^β diminue, alors $F > 0$,
- un flux nul correspond à une zone localement plate.

La superposition de $u(x)$ et de $F(x)$ montre que :

- le flux est nul là où $u(x) = 0$,

- le flux est négatif sur le flanc gauche de la bosse,
- le flux est positif sur le flanc droit,
- le flux s'annule au sommet du profil.

Cela montre que la diffusion agit comme un mécanisme de **transport de masse depuis le centre vers l'extérieur**.

1.9 Simulation en temps, snapshots, min(u), masse

```
[10]: # =====
# Simulation en temps pour un donné
# =====

# Copie de la condition initiale pour démarrer la simulation
u = u_init.copy()

# Définition des instants où on veut capturer des snapshots de la solution
times_snap = [0.0, T*0.01, T*0.05, T*0.1, T*0.2, T*0.5, T, 2*T, 5*T, 10*T]

# Conversion des temps en indices de pas de temps
k_snap = [int(t/dt) for t in times_snap]

# Liste pour stocker les snapshots de u(x,t) aux instants choisis
snapshots = [u_init.copy()] # on stocke aussi t=0

# Listes pour suivre l'évolution du minimum de u et de la masse totale
mins = [u.min()] # Minimum initial de u
mass = [np.trapz(u, x)] # Masse initiale (intégrale de u sur le
    ↪ domaine)
times = [0.0] # Liste des temps

# Boucle principale : itération sur tous les pas de temps
for k in range(1, M+1):
    # Avance d'un pas de temps avec le schéma numérique
    u = one_time_step(u, beta, dx, dt)

    # Si on est à un instant de snapshot, on sauvegarde u
    if k in k_snap:
        snapshots.append(u.copy())

    # Enregistrement du minimum de u à chaque pas de temps
    mins.append(u.min())

    # Calcul et enregistrement de la masse totale à chaque pas de temps
    mass.append(np.trapz(u, x))

    # Enregistrement du temps correspondant
    times.append(k*dt)
```

```

# =====
# Visualisation des snapshots de  $u(x,t)$ 
# =====
plt.figure(figsize=(7,4))

# Tracé de  $u(x,t)$  à différents instants
for t, u_snap in zip(times_snap, snapshots):
    plt.plot(x, u_snap, label=f"t = {t:.3f}")

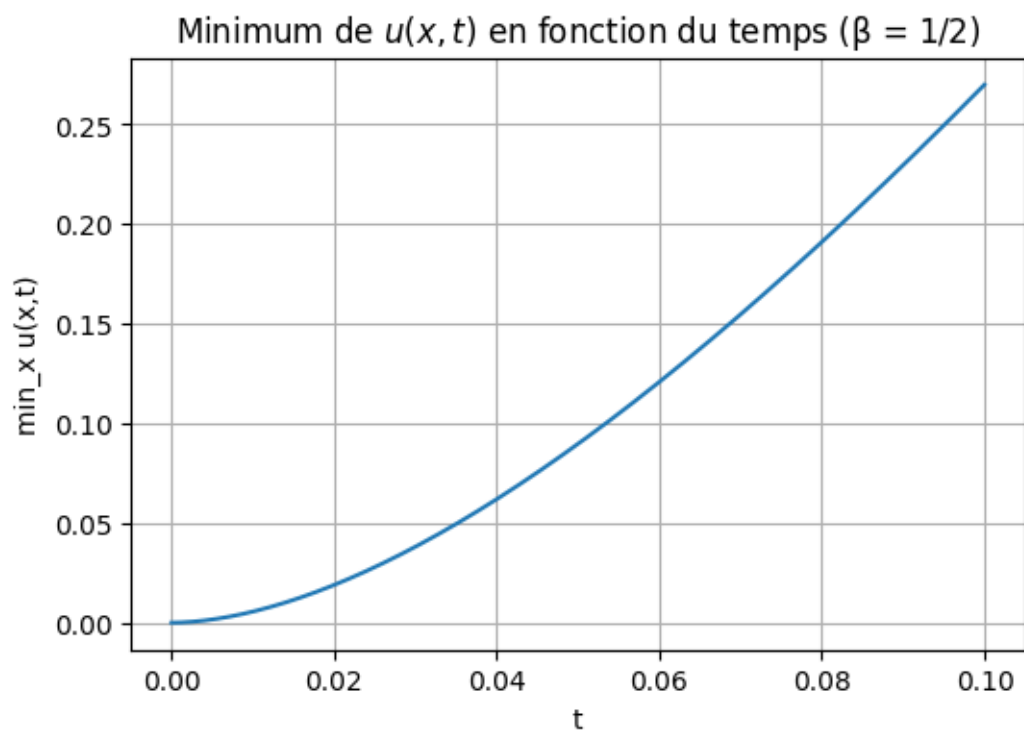
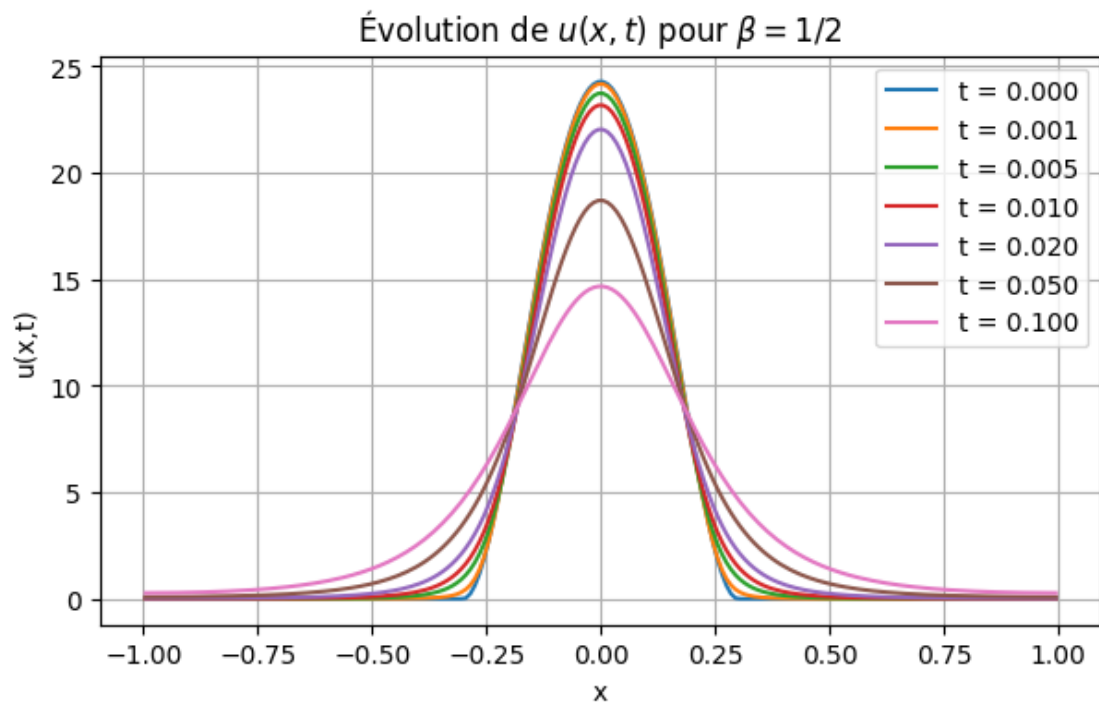
plt.xlabel("x")
plt.ylabel("u(x,t)")
plt.title(r"Évolution de  $u(x,t)$  pour  $\beta = 1/2$ ")
plt.legend()
plt.grid(True)
plt.show()

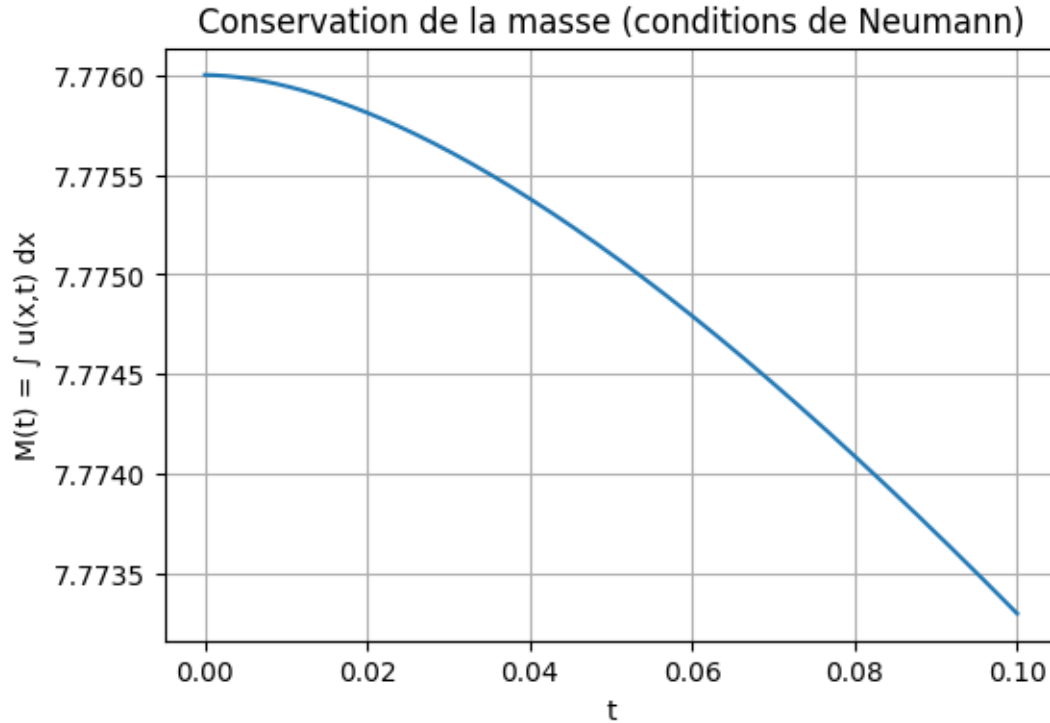
# =====
# Visualisation du minimum de  $u$  en fonction du temps
# =====
plt.figure(figsize=(6,4))
plt.plot(times, mins)
plt.xlabel("t")
plt.ylabel("min_x u(x,t)")
plt.title(r"Minimum de  $u(x,t)$  en fonction du temps ( $\beta = 1/2$ ")
plt.grid(True)
plt.show()

# =====
# Visualisation de la conservation de la masse
# =====
plt.figure(figsize=(6,4))
plt.plot(times, mass)
plt.xlabel("t")
plt.ylabel("M(t) =  $\int u(x,t) dx$ ")
plt.title("Conservation de la masse (conditions de Neumann)")
plt.grid(True)
plt.show()

# Affichage des valeurs finales pour vérification
print("Min(u) final =", mins[-1])
print("Masse initiale =", mass[0], "Masse finale =", mass[-1])

```





Min(u) final = 0.2700344228034075

Masse initiale = 7.776000524999999 Masse finale = 7.773300180771966

1.10 Commentaire : Évolution temporelle pour $\beta = \frac{1}{2}$

On effectue ensuite une simulation jusqu'au temps final $T = 0.1$, en sauvegardant des snapshots de la solution :

$$u(x, t) \quad \text{pour} \quad t \in [0, T].$$

1.10.1 Snapshots de la solution

La figure correspondante montre :

- une diminution progressive du maximum au centre,
- un élargissement du profil,
- l'apparition de valeurs positives hors du support initial.

Ce comportement est caractéristique de la diffusion rapide :

- la solution devient strictement positive pour tout $x \in [-1, 1]$ dès que $t > 0$,
- le support compact initial disparaît immédiatement.

1.10.2 Minimum de la solution

On trace l'évolution de :

$$m(t) = \min_x u(x, t).$$

Les observations sont :

- $m(0) = 0$ puisque $u_0(x) = 0$ en dehors de $[-0.3, 0.3]$,
- $m(t)$ augmente de façon monotone,
- aucun passage en valeurs négatives n'est observé.

Cela illustre :

- la **conservation de la positivité**,
- la tendance de la solution à se répartir uniformément sur le domaine.

1.10.3 Conservation de la masse

On calcule la masse au cours du temps :

$$M(t) = \int_{-1}^1 u(x, t) dx.$$

On observe que :

- $M(t)$ est pratiquement constante,
- les fluctuations sont de l'ordre de 10^{-4} .

Cela confirme que le schéma en volumes finis est bien conservatif sous conditions de Neumann.

La masse initiale étant de 7.776, la solution tend à long terme vers la solution stationnaire constante :

$$u(x, t) \rightarrow \frac{M}{2} \approx 3.888,$$

ce qui correspond à l'état d'équilibre attendu pour une diffusion sans pertes aux bords.

1.11 Vérification des conditions aux bords (Neumann)

```
[11]: # =====
# Vérification des conditions de Neumann
# =====

# Calcul de u^ en s'assurant que u reste positif
u_beta = np.maximum(u, 0.0)**beta

# Approximation de la dérivée x(u^ ) au bord gauche (x = -1)
# Différence finie progressive : (u_beta[1] - u_beta[0]) / dx
d_left = (u_beta[1] - u_beta[0]) / dx

# Approximation de la dérivée x(u^ ) au bord droit (x = +1)
# Différence finie rétrograde : (u_beta[-1] - u_beta[-2]) / dx
```



```

d_right = (u_beta[-1] - u_beta[-2]) / dx

# Affichage des dérivées aux bords pour vérifier les conditions de Neumann
# Si les conditions sont bien respectées, ces valeurs devraient être proches de 0
print("Approx x(u^ ) au bord gauche :", d_left)
print("Approx x(u^ ) au bord droit  :", d_right)

```

Approx x(u^) au bord gauche : 0.04123370858706288
 Approx x(u^) au bord droit : -0.041233708587051776

1.12 Commentaire : Vérification des conditions de Neumann

On calcule numériquement les dérivées discrètes de u^β près des bords :

$$\partial_x(u^\beta)(x_{1/2}) \approx \frac{(u^\beta)_1 - (u^\beta)_0}{dx},$$

$$\partial_x(u^\beta)(x_{N+1/2}) \approx \frac{(u^\beta)_{N-1} - (u^\beta)_{N-2}}{dx}.$$

Ces dérivées ne sont pas exactement nulles car :

- elles mesurent la pente **à l'intérieur du domaine**,
- alors que la condition de Neumann est imposée via les **flux aux interfaces extrêmes**.

Dans le schéma, la condition :

$$F_{1/2} = F_{N+1/2} = 0$$

est assurée en pratique par des cellules fantômes :

$$(u^\beta)_{-1} = (u^\beta)_0, \quad (u^\beta)_N = (u^\beta)_{N-1}.$$

Ainsi, même si les pentes internes ne sont pas nulles, le flux réel au bord est nul, ce qui garantit des conditions de Neumann correctes.

1.13 Comparaison de plusieurs (fast / linéaire / slow)

```

[12]: # =====
# Plusieurs , plusieurs temps t
# =====

# Liste des valeurs de  à tester (exposant dans u^ )
betas = [0.1, 0.5, 1.0, 2.0]      # on peut en mettre plus, mais ça fait vite
# beaucoup de courbes

# Liste des instants où on veut visualiser la solution

```

```

times_plot = [0.0, 0.005, 0.01, 0.02]    # instants voulus

# Conversion des temps en indices de pas de temps
k_plot = [int(t/dt) for t in times_plot]

# Détermination du nombre maximum de pas de temps à calculer
k_max = max(k_plot)

# Création d'une grille de 2x2 sous-graphiques (un par valeur de )
fig, axes = plt.subplots(2, 2, figsize=(10, 6), sharex=True, sharey=True)

# Conversion de la grille 2D en liste 1D pour faciliter l'itération
axes = axes.ravel()

# Boucle sur chaque valeur de
for ax, beta_val in zip(axes, betas):
    # Initialisation : on part de la condition initiale u pour ce
    u = u_init.copy()

    # Dictionnaire pour stocker les snapshots aux instants demandés
    snapshots = {0.0: u.copy()}

    # Évolution temporelle jusqu'au temps maximum demandé
    for k in range(1, k_max + 1):
        # Avance d'un pas de temps avec la valeur de  actuelle
        u = one_time_step(u, beta_val, dx, dt)

        # Si on est à un instant de snapshot, on sauvegarde u
        if k in k_plot:
            snapshots[k*dt] = u.copy()

    # Tracé des solutions aux différents instants pour ce
    for t in times_plot:
        ax.plot(x, snapshots[t], label=fr"t = {t:.3f}")

    # Ajout de la condition initiale en pointillés noirs pour référence
    ax.plot(x, u_init, 'k--', linewidth=1, label="u ")

    # Titre du sous-graphique avec la valeur de
    ax.set_title(fr"$\beta = {beta_val}$")
    ax.grid(True)

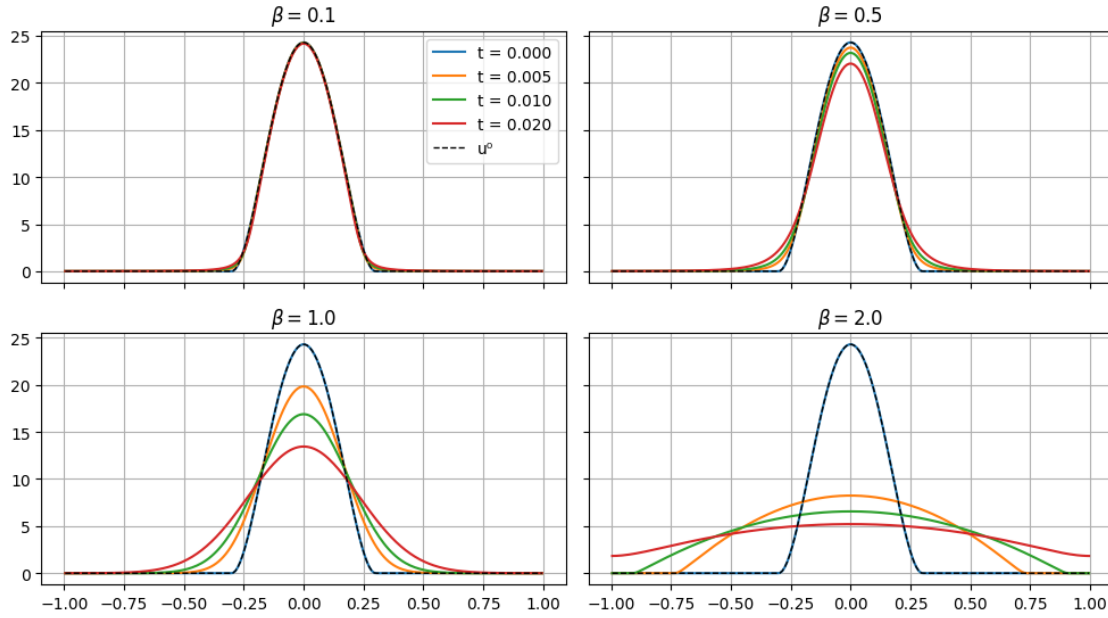
# Affichage de la légende uniquement sur le premier sous-graphique
axes[0].legend()

# Titre général de la figure
fig.suptitle("Évolution de u(x,t) pour différents  et temps t", y=1.02)

```

```
# Ajustement automatique de l'espacement entre les sous-graphiques
plt.tight_layout()
plt.show()
```

Évolution de $u(x,t)$ pour différents β et temps t



1.14 Commentaire : Influence du paramètre β

On étudie maintenant l'influence du paramètre β sur la diffusion, en comparant plusieurs valeurs :

$$\beta \in \{0.1, 0.5, 1, 2\}.$$

On trace les profils de solution $u(x,t)$ pour plusieurs instants, pour chaque valeur de β .

1.14.1 Trois régimes de diffusion

L'équation s'écrit :

$$u_t = (u^\beta)_{xx} = \partial_x(\beta u^{\beta-1} u_x).$$

Le coefficient de diffusion effectif est donc :

$$D(u) = \beta u^{\beta-1}.$$

Selon la valeur de β , ce coefficient varie fortement :

1.14.2 Régime $\beta < 1$: diffusion rapide

Exemples : $\beta = 0.1$ et $\beta = 0.5$.

On observe que :

- la bosse centrale s'aplatit très rapidement,
- la solution devient strictement positive sur tout le domaine,
- les profils à différents temps se séparent fortement.

Interprétation :

Lorsque $\beta < 1$, on a :

$$u^{\beta-1} \rightarrow +\infty \quad \text{quand } u \rightarrow 0.$$

Donc la diffusion est **très intense dans les zones de faible densité** : la masse s'étale immédiatement dans les régions initialement vides.

1.14.3 Régime $\beta = 1$: diffusion linéaire

Cas $\beta = 1$:

$$u_t = u_{xx}.$$

- La diffusion est homogène,
- la bosse se lisse progressivement,
- le comportement est intermédiaire entre les deux autres régimes.

C'est le modèle classique de la chaleur.

1.14.4 Régime $\beta > 1$: diffusion lente

Exemple : $\beta = 2$.

On observe que :

- les profils restent proches de u_0 ,
- la diffusion est faible,
- les différentes courbes temporelles sont presque confondues.

Interprétation :

Quand $\beta > 1$,

$$u^{\beta-1} \rightarrow 0 \quad \text{quand } u \rightarrow 0.$$

Donc la diffusion est **très faible dans les zones peu denses** : la masse ne pénètre que très lentement dans les régions où u est petit.

1.14.5 Conclusion intermédiaire

Le paramètre β contrôle fortement la dynamique :

- $\beta < 1$: diffusion rapide, propagation instantanée,
- $\beta = 1$: diffusion standard,
- $\beta > 1$: diffusion lente.

Les figures numériques confirment parfaitement ces comportements théoriques.

1.15 8. Influence du nombre de points N

```
[13]: # =====  
# Influence combinée de  et N  
# =====  
  
# Valeurs de  à tester : diffusion rapide (0.5), linéaire (1.0), lente (2.0)  
betas = [0.5, 1.0, 2.0]          # fast, linéaire, slow  
  
# Valeurs de N (nombre de points de discrétisation) à tester  
Ns     = [50, 100, 200]  
  
# Instants où on veut capturer la solution  
times_plot = [0.0, 0.005, 0.01, 0.02]  
  
# Conversion des temps en indices de pas de temps  
k_plot = [int(t/dt) for t in times_plot]  
  
# Nombre maximum de pas de temps à calculer  
k_max = max(k_plot)  
  
def run_with_N_beta_snapshots(N, beta, dt, times_plot):  
    """  
    Fonction qui simule l'équation pour un N et un  donnés,  
    et retourne les snapshots aux instants demandés.  
    """  
    # Longueur du domaine spatial  
    L = 2.0  
  
    # Calcul du pas d'espace en fonction de N  
    dx = L / N  
  
    # Création de la grille spatiale avec N points (centres des cellules)  
    xN = -1.0 + (np.arange(N) + 0.5)*dx  
  
    # Condition initiale sur cette grille  
    uN = u0(xN)  
  
    # Dictionnaire pour stocker les snapshots
```

```

snapshots = {0.0: uN.copy()}

# Conversion des temps en indices pour cette simulation
k_plot = [int(t/dt) for t in times_plot]
k_max = max(k_plot)

# Boucle temporelle
for k in range(1, k_max + 1):
    # Avance d'un pas de temps
    uN = one_time_step(uN, beta, dx, dt)

    # Sauvegarde si on est à un instant de snapshot
    if k in k_plot:
        snapshots[k*dt] = uN.copy()

return xN, snapshots

# Création d'une grille de sous-graphiques : une ligne par , une colonne par N
fig, axes = plt.subplots(len(betas), len(Ns), figsize=(14, 8), sharex=True,
    ↳sharey=True)

# Si len(betas)=1 ou len(Ns)=1, axes peut être 1D, on aplatit pour simplifier
axes = np.atleast_2d(axes)

# Double boucle : on teste toutes les combinaisons ( , N)
for i, beta_val in enumerate(betas):
    for j, N_val in enumerate(Ns):
        # Sélection du sous-graphique correspondant
        ax = axes[i, j]

        # Simulation avec les paramètres (N_val, beta_val)
        xN, snaps = run_with_N_beta_snapshots(N_val, beta_val, dt, times_plot)

        # Tracé des snapshots aux différents instants
        for t in times_plot:
            ax.plot(xN, snaps[t], label=fr"t = {t:.3f}")

        # Titre de colonne : affiche N en haut de chaque colonne
        if i == 0:
            ax.set_title(fr"N = {N_val}")

        # Titre de ligne : affiche à gauche de chaque ligne
        if j == 0:
            ax.set_ylabel(fr"$\beta = {beta_val}")

ax.grid(True)

```

```

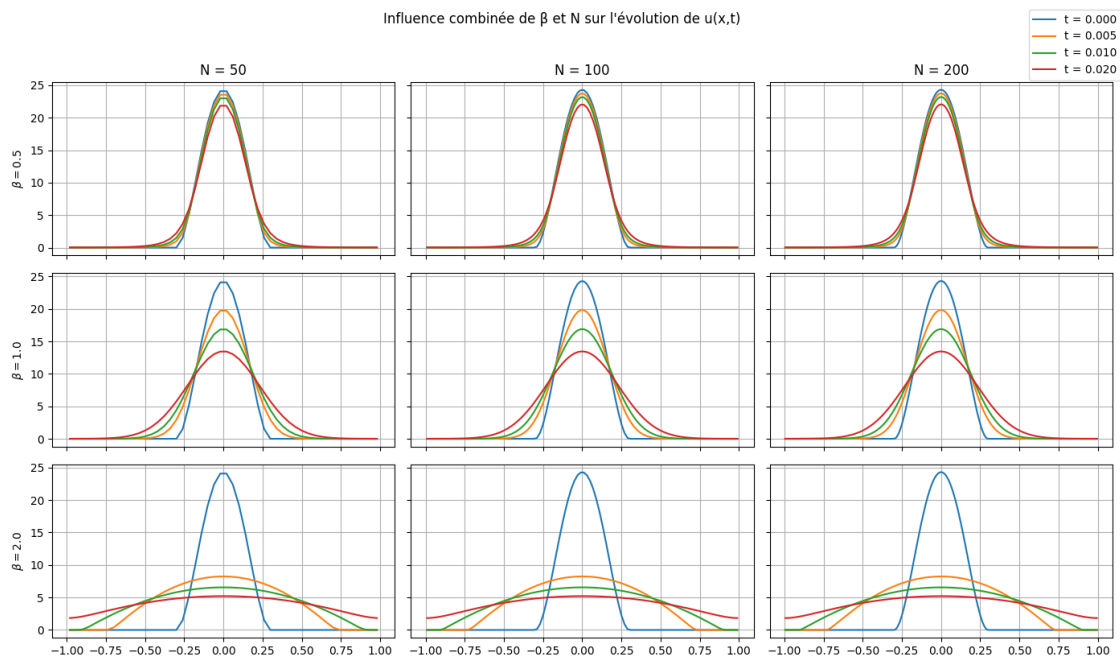
# Récupération des handles et labels pour une légende commune
handles, labels = axes[0,0].get_legend_handles_labels()

# Légende commune placée en haut à droite de la figure
fig.legend(handles, labels, loc="upper right")

# Titre général de la figure
fig.suptitle("Influence combinée de  $\beta$  et N sur l'évolution de  $u(x,t)$ ", y=1.02)

# Ajustement automatique de l'espacement
plt.tight_layout()
plt.show()

```



1.16 Commentaire : Influence du nombre de points N

On étudie maintenant la dépendance numérique en fonction du maillage, en prenant :

$$N \in \{50, 100, 200\},$$

avec $\beta = 0.5$ et $T = 0.02$.

1.16.1 Observations numériques

On compare les solutions obtenues sur ces trois maillages.

On observe que :

- les courbes se superposent presque,
- les profils deviennent légèrement plus lisses lorsque N augmente,
- la forme globale de la solution reste inchangée.

1.16.2 Interprétation

Quand N augmente, la taille des cellules dx diminue :

$$dx = \frac{2}{N}.$$

La convergence des profils lorsque N augmente montre que :

- le schéma est **stable**,
- le schéma est **convergent**,
- $N = 200$ est suffisant pour décrire fidèlement la solution.

On peut donc conclure que les résultats observés précédemment **ne sont pas des artefacts numériques**, mais bien des effets physiques de l'équation étudiée.

1.17 Influence combinée de β et de N

On combine maintenant les deux paramètres :

$$\beta \in \{0.5, 1, 2\}, \quad N \in \{50, 100, 200\}.$$

Chaque ligne correspond à une valeur de β , chaque colonne à une valeur de N .

1.17.1 Analyse

Influence de N (par colonnes) Pour un β fixé :

- augmenter N ne modifie pas la dynamique observée,
- la solution devient plus régulière,
- les profils convergent.

Cela confirme la **consistance numérique** du schéma.

Influence de β (par lignes) Pour un N fixé :

- $\beta = 0.5$: diffusion rapide,
- $\beta = 1$: diffusion intermédiaire,
- $\beta = 2$: diffusion lente.

Cela montre que :

β est un paramètre clef du modèle.

Il ne règle pas seulement la vitesse de diffusion, mais aussi la structure de la solution.

1.17.2 Conclusion de la double étude

L'influence de β est d'origine physique,
l'influence de N est d'origine numérique.

Les résultats sont robustes vis-à-vis de la discrétisation $\rightarrow$
les tendances observées sont donc fiables.

1.18 Conclusion générale

Ce travail numérique a mis en évidence :

- le bon fonctionnement du schéma implicite,
- la conservation de la masse,
- la positivité de la solution,
- l'influence déterminante du paramètre β ,
- la stabilité numérique du schéma vis-à-vis du maillage.

Les simulations confirment les trois régimes théoriques :

Valeur de	Type de diffusion
< 1	diffusion rapide
$= 1$	diffusion classique
> 1	diffusion lente

L'étude numérique est cohérente avec la théorie et montre la capacité du schéma à reproduire fidèlement des phénomènes non linéaires complexes.

Ce modèle constitue ainsi un cadre simplifié permettant de comprendre qualitativement les mécanismes de diffusion observés dans le contexte tumoral.